

▼ ** SAA - ACTIVIDAD 3.18 - ALVARO FERNANDEZ**

Resuelve el problema de clasificación de las enfermedades del corazón con un clasificador AdaBoostClassifier.

Utiliza validación simple con un 80% de datos para entrenamiento.

Utiliza GridSearchCV para encontrar valores óptimos para sus hiperparámetros `n_estimators` y `learning_rate`.

Por defecto, AdaBoostClassifier utiliza como clasificadores base árboles de clasificación de profundidad máxima 1.

El clasificador base se especifica con el parámetro `base_estimator`, cuyo valor por defecto es `DecisionTreeClassifier(max_depth=1)`.

Haz que GridSearchCV pruebe con árboles de clasificación de profundidades 1 y 2, para seleccionar el valor con el que se obtienen mejores resultados.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"

columnas = [
    "age", "sex", "cp", "trestbps", "chol", "fbs", "restecg", "thalach",
    "exang", "oldpeak", "slope", "ca", "thal", "target"
]

df = pd.read_csv(url, names=columnas, na_values='?')
df = df.dropna().copy()

df["target"] = df["target"].astype(int)
df["target"] = (df["target"] > 0).astype(int)

x = df.drop("target", axis=1)
y = df["target"]

RANDOM_SEED = 33

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=RANDOM_SEED, stratify=y
)
```

```
clf = AdaBoostClassifier(random_state=RANDOM_SEED)
clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

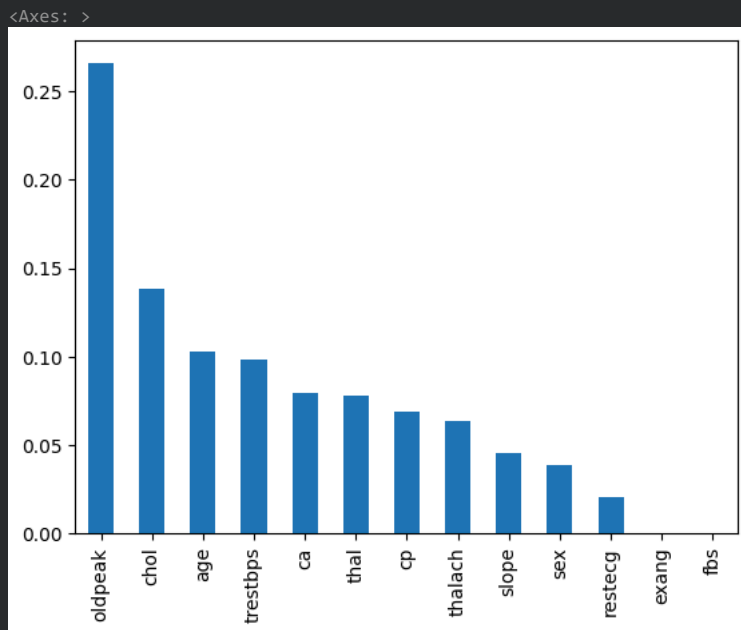
print("AdaBoost")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
AdaBoost
Accuracy: 0.85
```

	precision	recall	f1-score	support
0	0.81	0.94	0.87	32
1	0.91	0.75	0.82	28
accuracy			0.85	60
macro avg	0.86	0.84	0.85	60
weighted avg	0.86	0.85	0.85	60

▼ Pesos:

```
pd.Series(clf.feature_importances_,
index=clf.feature_names_in_.nlargest(clf.feature_names_in_.shape[0]).plot(kind='bar')
```



✓ Aplicar Gridsearch:

- En `estimator` ponemos los 2 arboles con profundidad 1 y 2.

```
param_dist = {
    'estimator': [DecisionTreeClassifier(random_state=RANDOM_SEED, max_depth=1),
                  DecisionTreeClassifier(random_state=RANDOM_SEED, max_depth=2)],
    #'estimator': [DecisionTreeClassifier(random_state=RANDOM_SEED)],
    #'estimator__max_depth': [1, 2],
    'n_estimators': [25, 50, 100, 200], # Valor por defecto: 50.
    'learning_rate': [0.25, 0.5, 1, 2, 4, 8], # Valor por defecto: 1. "trade-off between the learning_rate and n_estimators par
}

grid_clf = GridSearchCV(
    estimator=AdaBoostClassifier(random_state=RANDOM_SEED),
    param_grid=param_dist,
    cv=5
)

grid_clf.fit(x, y)

print("Mejores hiperparámetros:")
print(grid_clf.best_params_)
print(f"\nMejor accuracy: {grid_clf.best_score_}")
```

Mejores hiperparámetros:
{'estimator': DecisionTreeClassifier(max_depth=1, random_state=33), 'learning_rate': 0.25, 'n_estimators': 25}

Mejor accuracy: 0.8348022598870056

✓ - Entrenar el mejor modelo:

```
clf = grid_clf.best_estimator_
clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

print("accuracy score:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

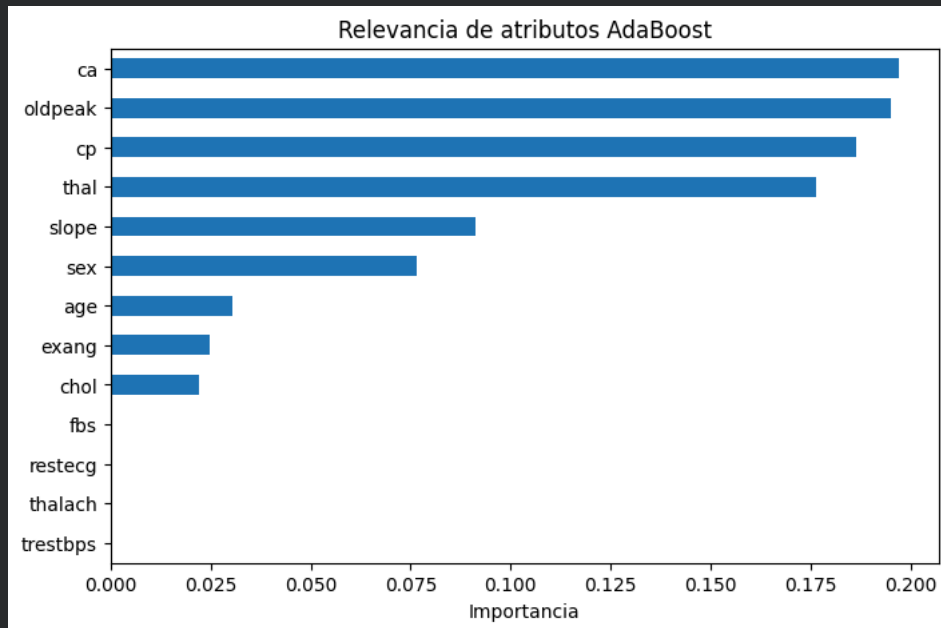
```
accuracy score: 0.8833333333333333
precision    recall  f1-score   support

         0         0.86      0.94      0.90         32
         1         0.92      0.82      0.87         28

   accuracy          0.88         60
  macro avg          0.89         60
 weighted avg          0.89         60
```

- Pesos:

```
pd.Series(clf.feature_importances_, index=clf.feature_names_in_)\
.sort_values(ascending=True)\
.plot(kind="barh", figsize=(8, 5), title="Relevancia de atributos AdaBoost")
plt.xlabel("Importancia")
plt.show()
```



- Despues de aplicar Gridsearch hemos pasado de un accuracy de 0.85 a 0.88
- A destacar que el atributo ca pasa de mitad de tabla aproximadamente en cuanto a importancia a ser el que más peso tiene.